

TRIOPATH

DREAM DISCOVER DECIDE

Employee Leave Management System

Secure Enterprise Leave Automation API

A production-ready REST API built on ASP.NET Core that automates employee leave workflows, enforces enterprise business rules, secures access via JWT authentication, and streamlines HR approval processes end-to-end.

BACKEND ENGINEERING PORTFOLIO

PRESENTER: [YOUR NAME]

The Business Problem

Traditional HR leave management relies on manual, error-prone processes that create bottlenecks, expose security gaps, and reduce workforce transparency. These pain points compound across growing organisations.

Manual Approvals

Leave requests processed via email or paper, causing delays of days or weeks with no audit trail.

Poor Tracking

No centralised balance ledger – HR manually counts days taken, leading to frequent miscalculations.

Overlapping Requests

Multiple employees requesting the same dates with zero conflict detection or team-coverage validation.

Security Gaps

No role enforcement – any employee can view or modify another's leave records in shared systems.

No Automation

Approval workflows are entirely manual – managers must chase requests across inboxes and spreadsheets.

Project Objectives

The Employee Leave Management System is engineered to solve every identified pain point through a secure, automated, and auditable API layer that integrates cleanly into existing enterprise HR infrastructure.



Each objective maps directly to a core API module – ensuring that business goals and technical deliverables are fully aligned and measurable from day one of deployment.

Technology Stack



ASP.NET Core / .NET 7

High-performance, cross-platform web API framework powering all controllers, middleware, and dependency injection.



Serilog + Swagger

Structured file-based logging via Serilog and interactive API documentation via Swagger/OpenAPI for rapid testing.



SQL Server + EF Core

Relational data store with Entity Framework Core ORM for type-safe queries, migrations, and schema management.



xUnit + Moq

Unit and integration testing using xUnit test runner with Moq for service mocking and in-memory DB for isolation.

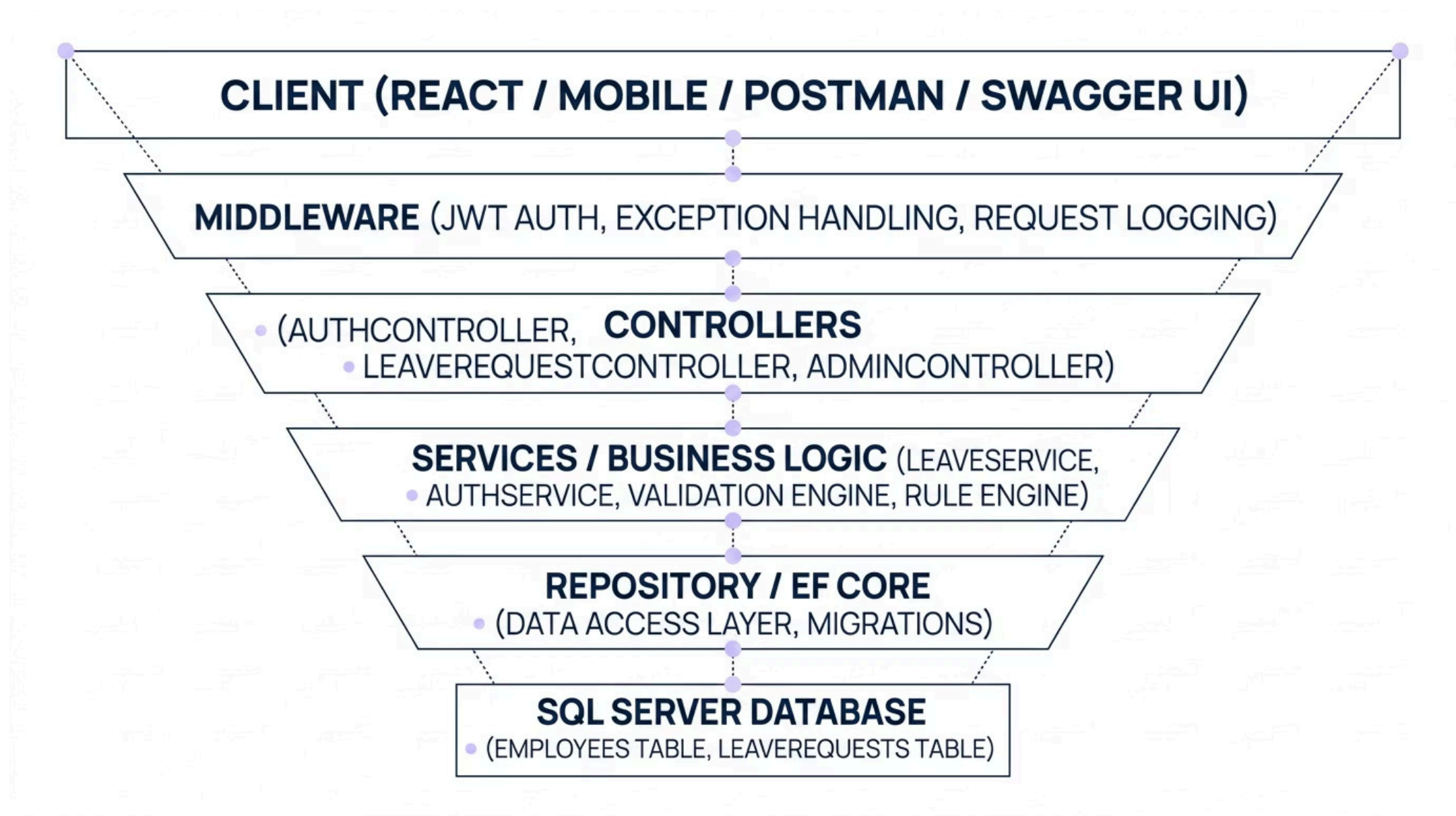


JWT Authentication

Stateless token-based auth using JSON Web Tokens – signed with HS256 for secure, scalable session management.

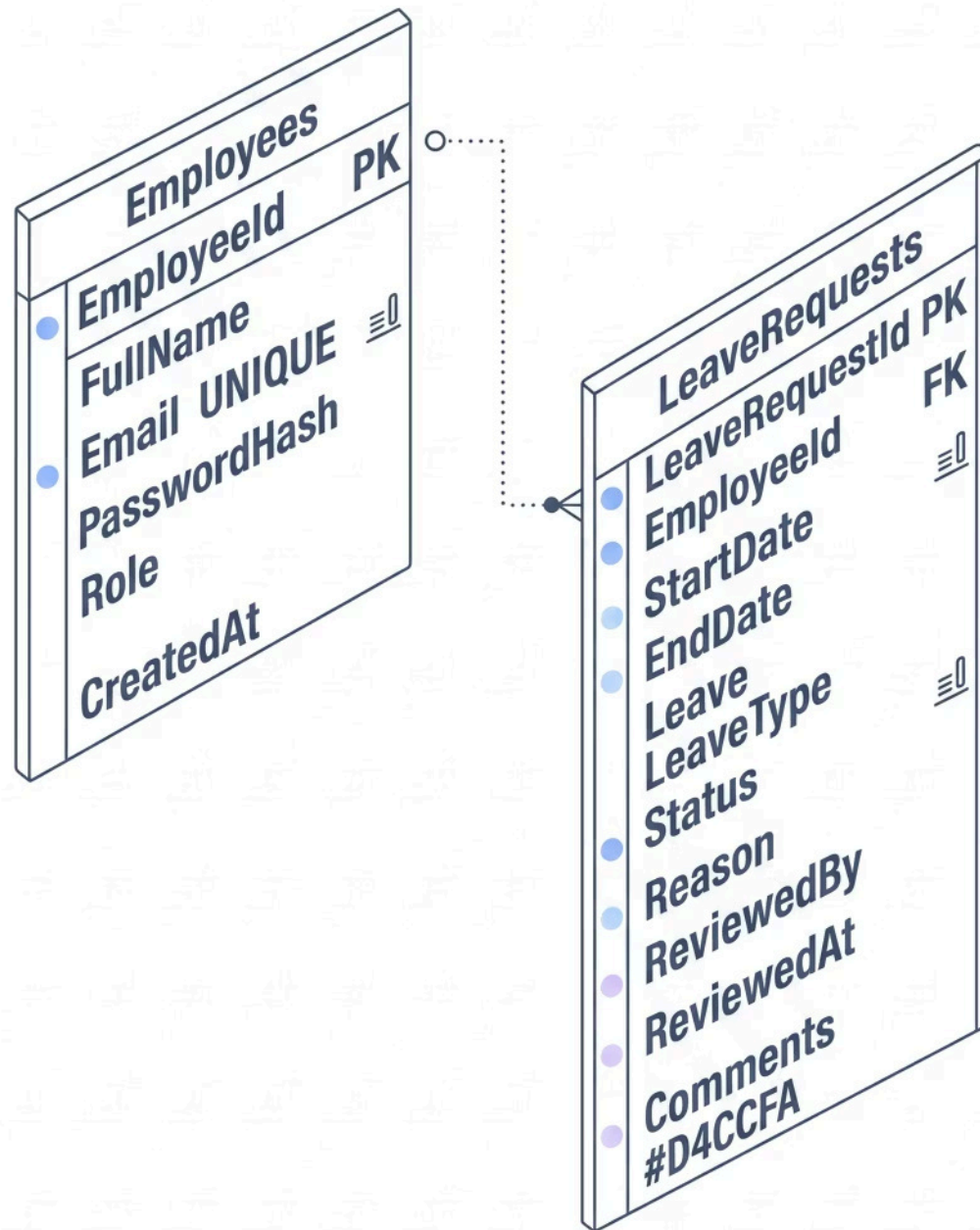
System Architecture

The API follows a clean, layered architecture pattern – separating concerns across presentation, business logic, and data access layers, with cross-cutting middleware for exception handling and logging.



Each layer is strictly decoupled – controllers delegate to services, services call repositories, and middleware intercepts every request transparently, ensuring clean separation of concerns throughout.

Database Design



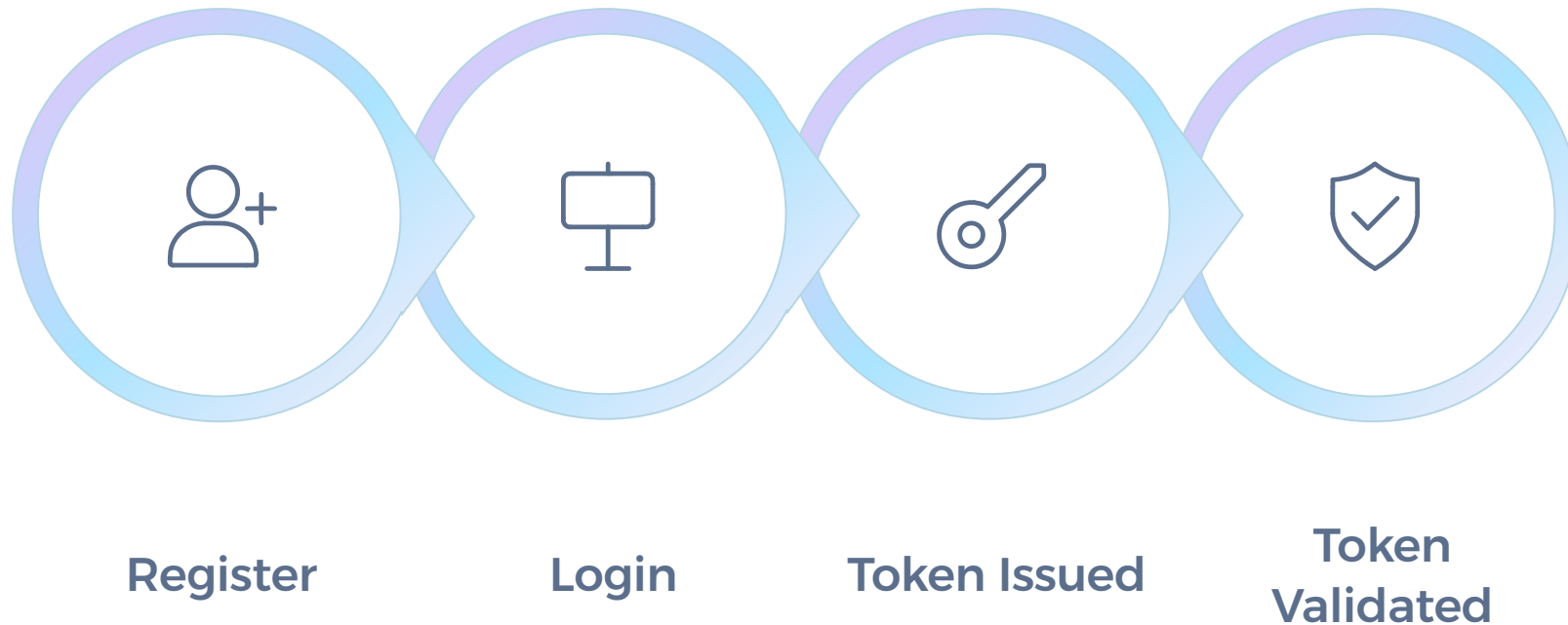
Schema Highlights

- **EmployeeId** – GUID primary key for globally unique identity
- **PasswordHash** – BCrypt-hashed, never stored as plain text
- **Role** – Enum: Employee | Manager | Admin
- **Status** – Enum: Pending | Approved | Rejected
- **ReviewedBy** – FK to manager who actioned the request

i Indexes are applied on **Email**, **EmployeeId**, and **Status** columns to optimise frequent query patterns.

Authentication System

All API access is governed by a stateless JWT-based authentication flow. Passwords are never stored in plain text – BCrypt hashing is applied before persistence, and tokens are signed with a secret key and carry an expiry claim.



Token Payload Claims

- `sub` – EmployeeId (GUID)
- `role` – Employee | Manager | Admin
- `exp` – Expiry timestamp (configurable)

Security Controls

- BCrypt work factor: 12 rounds minimum
- Secret key stored in `appsettings.json`
- 401 returned on invalid or expired tokens

Role-Based Access Control (RBAC)

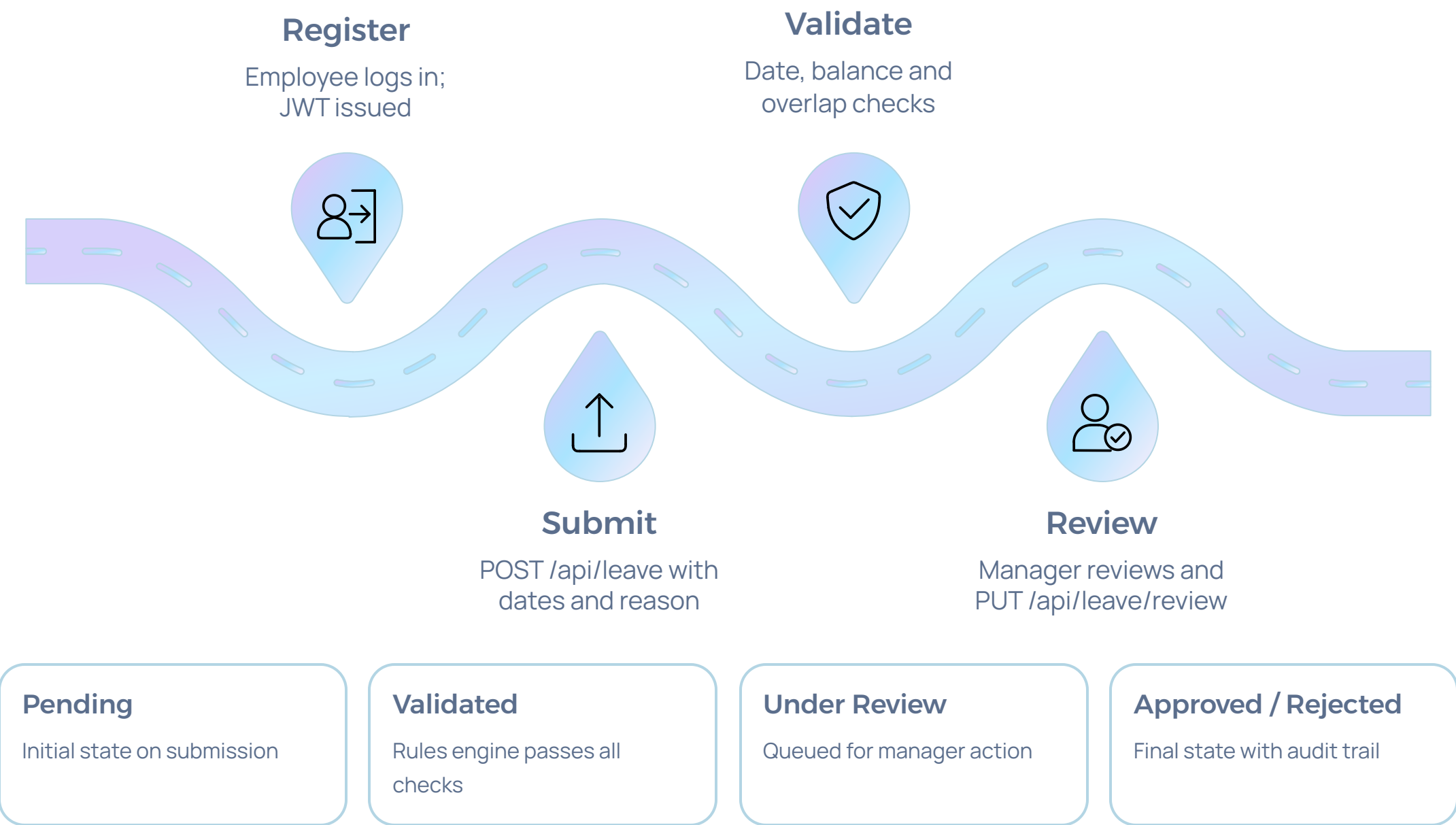
Three distinct roles govern what each user can see and do within the system. Roles are embedded in the JWT and enforced at the controller level via ASP.NET Core policy attributes – no role elevation is possible at runtime.

	Employee	Manager	Admin
Submit Leave Request	YES Access Granted	YES Access Granted	YES Access Granted
View Own Requests	NO	YES Access Granted	YES Access Granted
View All Requests	YES	NO	NO
Review and Approve/Reject Requests	NO	YES	NO
Manage Employees	NO	NO	NO
Access Admin Dashboard	NO	NO	NO

- ❑ Role enforcement uses `[Authorize(Roles = "Manager,Admin")]` policy decorators on each controller action – unauthorised access returns HTTP 403 Forbidden.

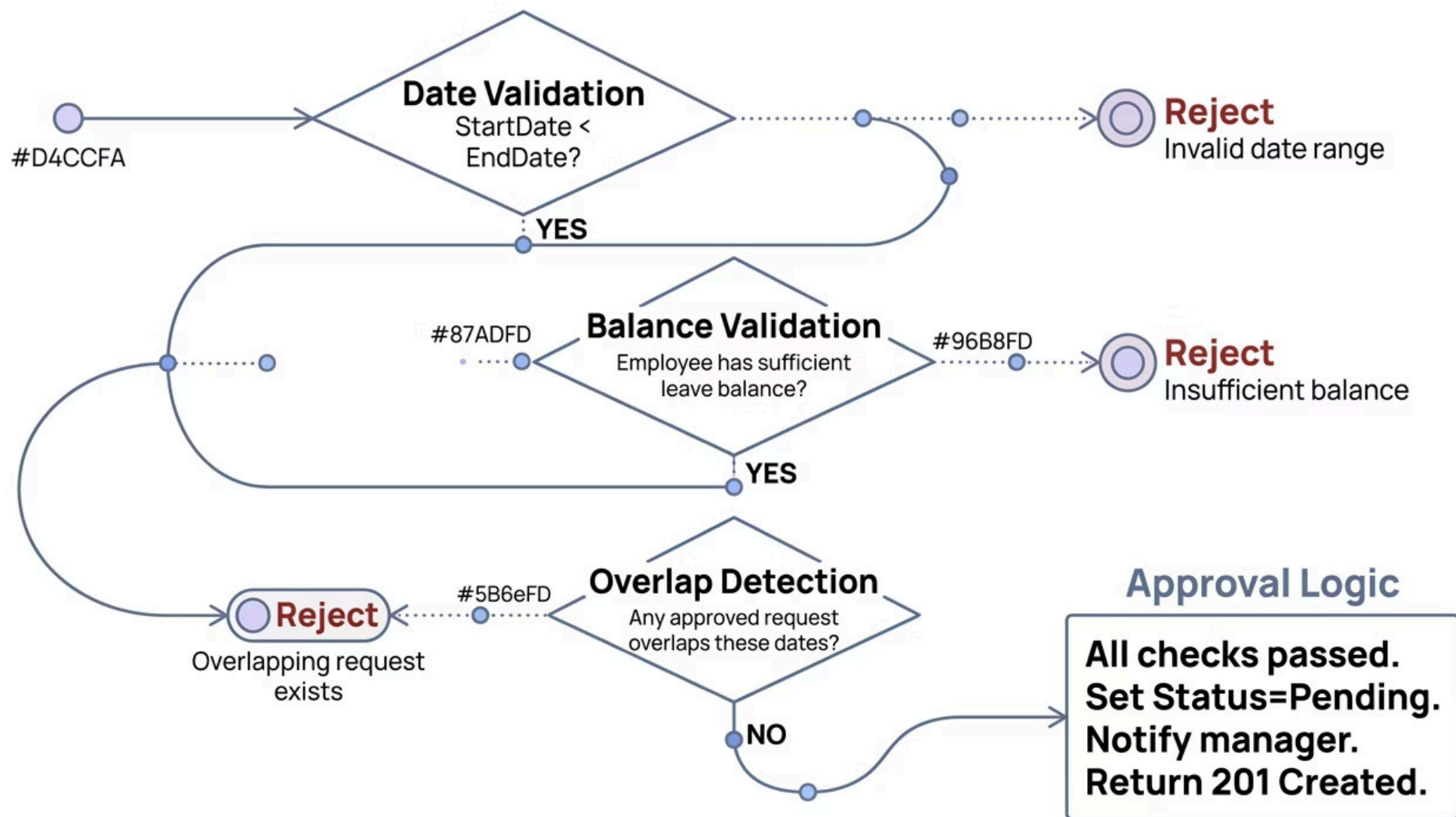
Leave Request Lifecycle

Every leave request follows a strict, auditable state machine – from initial submission through validation, manager review, and final status resolution. Each state transition is logged with timestamps and actor identity.



Business Rule Engine

Before any leave request enters the approval queue, it must pass a multi-stage validation pipeline. The rules engine is invoked synchronously within the service layer and returns structured error responses for any violation.



- ✓ All validation errors are returned as structured `ProblemDetails` JSON responses with field-level messages – making the API easy to integrate with any frontend client.

API Endpoints

Authentication Endpoints

Method	Endpoint	Access
POST	/api/auth/register	Public
POST	/api/auth/login	Public

Leave Request Endpoints

Method	Endpoint	Access
POST	/api/leave	Employee
GET	/api/leave/my	Employee
GET	/api/leave/all	Manager+
PUT	/api/leave/review/{id}	Manager+
DELETE	/api/leave/{id}	Admin

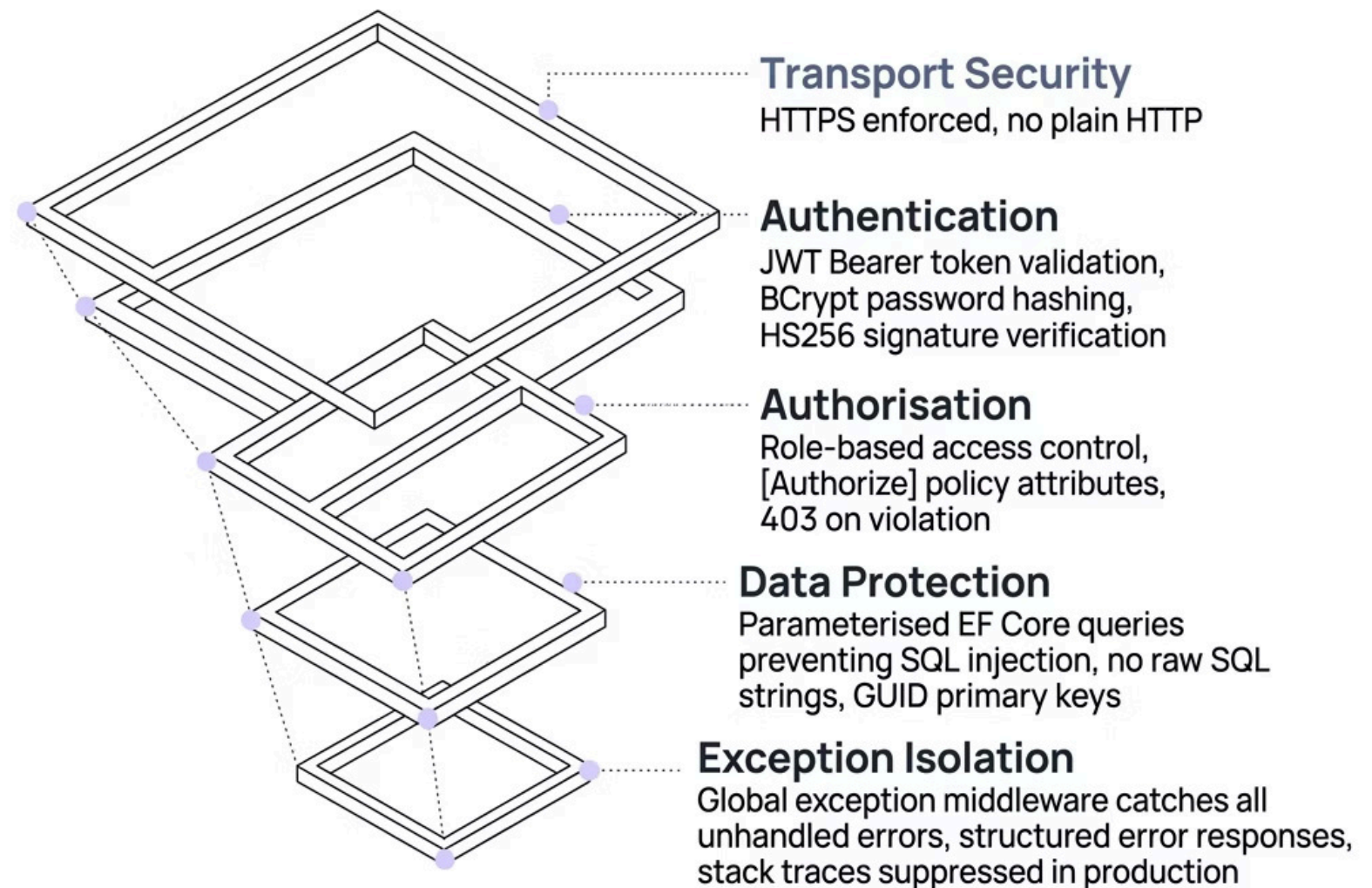
Response Standards

- **200 OK** – successful retrieval
- **201 Created** – resource created
- **400 Bad Request** – validation failure
- **401 Unauthorised** – missing/invalid JWT
- **403 Forbidden** – insufficient role
- **404 Not Found** – resource missing
- **500 Internal Server Error** – caught by middleware

 All endpoints documented via **Swagger UI** at /swagger – fully interactive for testing without Postman.

Security Architecture

Security is enforced in concentric layers — from transport through authentication, authorisation, data integrity, and error isolation. No single point of failure can expose sensitive employee data.



⚠ Secret keys and connection strings must **never** be committed to version control. Use environment variables or Azure Key Vault in production deployments.

Testing Strategy

The project includes a dedicated test suite covering all critical service-layer logic. Tests are fully isolated – no live database connections are required – using xUnit, Moq, and EF Core's in-memory provider.

1	Register – Duplicate Email Asserts 409 Conflict when email already exists in DB
2	Login – Invalid Credentials Asserts 401 Unauthorised on wrong password hash
3	Submit – Overlap Detection Asserts 400 when date range collides with existing request
4	Submit – Insufficient Balance Asserts 400 when requested days exceed remaining balance
5	Review – Role Enforcement Asserts 403 when Employee role attempts to approve requests
6	Happy Path – Full Flow Asserts 201 on valid submission and 200 on approval
7	Token – Expiry Validation Asserts 401 on expired JWT token presented to protected route

Testing Tools

- **xUnit** – primary test runner and assertion library
- **Moq** – mocking service and repository interfaces
- **EF Core InMemory** – isolated DB context per test
- **WebApplicationFactory** – integration test host

✔ 7 test cases cover all critical paths – achieving high confidence across auth, validation, and role enforcement scenarios.

Logging & Monitoring

Serilog provides structured, file-based logging with rich contextual metadata on every request, error, and business event. Log output is designed for both human debugging and machine parsing in production pipelines.

What Gets Logged

→ HTTP Request / Response

Method, path, status code, and response time on every API call

→ Authentication Events

Login attempts, token issuance, and failed auth with IP context

→ Business Rule Violations

Every validation failure logged with employee ID and rule name

→ Unhandled Exceptions

Full stack trace captured by middleware — never leaked to client

Log Configuration

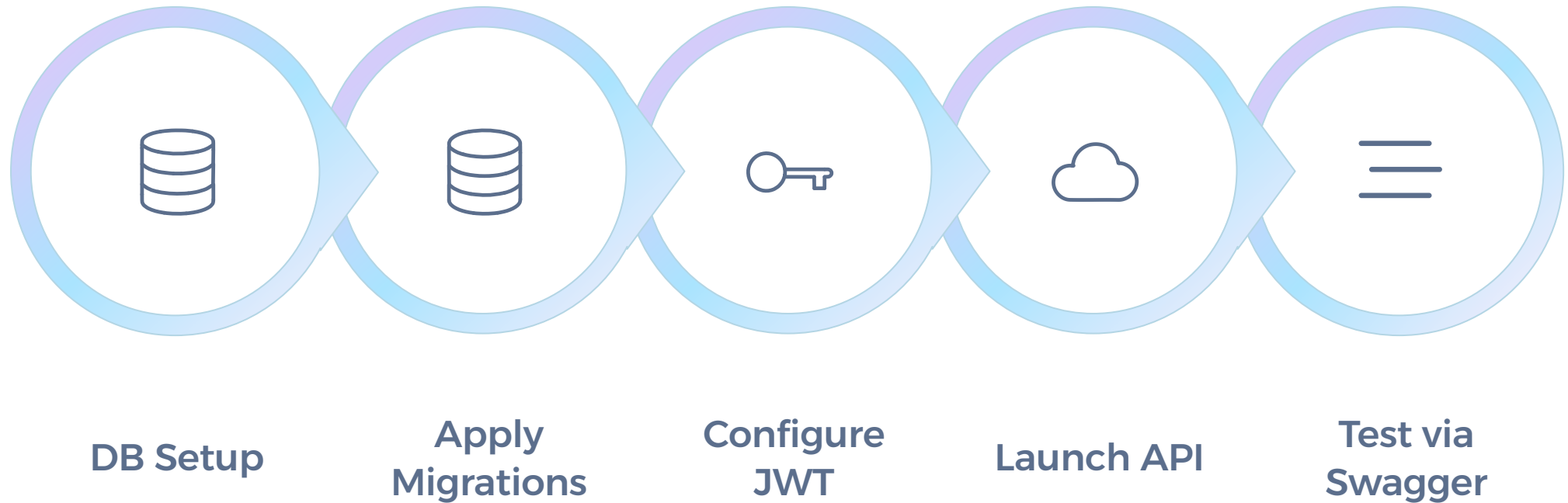
- **Sink:** Rolling file — `logs/app-.log`
- **Format:** JSON structured output
- **Levels:** Information | Warning | Error | Fatal
- **Enrichers:** Machine name, thread ID, correlation ID

Future Integrations

- Seq log viewer for real-time dashboard
- Azure Application Insights sink
- Elastic Stack (ELK) pipeline

Deployment & Setup

The system is designed for straightforward local and cloud deployment. All configuration is externalised into `appsettings.json` – making environment promotion from development to production clean and auditable.



i Use `dotnet user-secrets` or environment variables to override sensitive settings locally. Never commit secrets to Git. Add `appsettings.Production.json` to `.gitignore`.

Business Impact

Deploying the Employee Leave Management System delivers measurable operational improvements across HR efficiency, data accuracy, security posture, and employee satisfaction – with benefits compounding as headcount scales.

80%

Faster HR Processing

Automated approval workflows reduce average leave decision time from days to minutes

95%

Error Reduction

Business rule engine eliminates manual miscalculation of leave balances and overlapping approvals

100%

Audit Coverage

Every state transition is logged with actor identity, timestamp, and reason – full compliance trail

0

Unauthorised Access

RBAC + JWT ensures zero privilege escalation – employees can only view and submit their own data

Beyond metrics, the system positions HR teams to focus on strategic people operations rather than administrative overhead – a measurable quality-of-work improvement for both HR staff and employees alike.

Future Roadmap

Planned Enhancements — Phase 2 & Beyond

